Cipherbase: an LSM-Tree Storage Engine from Scratch

Mahesh Thakur

October 2026

Overview

Cipherbase is a log-structured merge-tree (LSM) key-value store implemented from scratch in Go - no storage dependencies, ~1,400 lines of engine code. LSM trees are the storage architecture behind RocksDB, Cassandra, ScyllaDB, HBase and LevelDB. This report covers the design, the crash-safety mechanics, the measured performance, and the two defects that testing and benchmarking surfaced.

Design

Write path

Every write is appended to a write-ahead log (WAL) before touching the memtable. Each WAL record is length-prefixed and CRC32-checksummed. The memtable is a skip list (expected O(log n) insert/lookup, ordered iteration). When the memtable exceeds 1 MiB it is flushed to an immutable sorted string table (SSTable) at Level 0 and the WAL is reset.

SSTable layout

```
data:   [klen u16][vlen u32][tomb u8][key][value] ...   (sorted by key)
index:  [klen u16][offset u64][key] ...                 (sparse: every 16th key)
bloom:  bitset + parameters
footer: [indexOff u64][bloomOff u64][count u64]
```

A point read binary-searches the sparse index for the covering block, then walks entries until the key is found or passed. The bloom filter (10 bits/key, 7 hash functions via double hashing of FNV-1a) is consulted first: a negative answer skips the file entirely. Measured false-positive rate: **0.97% at a 1% target** (test asserts < 2% over 100k probes).

Compaction

When Level 0 accumulates 4 SSTables they are merged with Level 1 into a new L1 file. Merge semantics: newest write wins; tombstones (delete markers) are dropped at L1 because no older level exists that could resurrect the key. A `manifest.json` records live files; recovery scans the directory, ignoring files not in the manifest (crash mid-compaction leaves orphans, which are harmless and unreferenced).

Range scans: the 260x fix

The first `Scan` implementation merged **every SSTable in full** and filtered by key afterwards - O(dataset) per query, measured at **113 ms** for a 100-key scan over 200k keys. The fix: each level opens an iterator positioned with its sparse index, and a k-way merge (memtable, L0 newest-first, L1) emits keys in order, stopping after `limit` live entries. Newer sources win duplicates; tombstones suppress. Measured after the fix: **0.43 ms** per 100-key scan - a **~260x speedup**. A second bug of the same family (reads bounded by file size instead of the index offset, leaking footer bytes into scans) was caught by the scan-ordering test.

Crash safety

The WAL replay test kills the process state (close without flush) and reopens: every acknowledged write must be readable. The WAL reader tolerates a torn tail: a partially written final record (the

expected outcome of a crash mid-append) fails its CRC and is discarded; all complete records before it replay cleanly.

Measured performance

Method: single goroutine, sequential calls, in-process engine API (no HTTP), fixed seed 42, 200k random 16-byte keys, 100-byte values, linux/amd64, Go 1.26.6. Reproduce with `go run ./cmd/bench`.

Operation	Result
Write	153,735 writes/s (6.5 µs/op), 1.30s total for 200k keys
Point read	28,110 reads/s (35.6 µs/op), 200k reads
Range scan, 100 keys	0.43 ms/op, 1,000 scans
Bloom filter	0.97% false positives at 1% target

Read latency is dominated by opening the SSTable file per lookup; an FD/block cache is the next optimization and was deliberately left out to keep the read path legible.

Testing

Nine tests, all passing under `go test -race`:

- put/get/delete round trips, including overwrite ordering
- memtable shadows SSTable after flush
- delete produces a tombstone that survives flush and compaction
- compaction drops shadowed keys and tombstones (file content inspected)
- crash recovery replays the WAL to the exact pre-crash state
- torn WAL tail is discarded without failing recovery
- bloom filter false-positive rate within 2x of target
- scan returns sorted, newest-wins results across memtable + L0 + L1
- manifest recovery ignores unreferenced SSTable files

Limitations

- Two levels, not a full leveled cascade; the merge policy is the teachable core
- No FD or block cache
- Compaction is size-triggered on L0 count only
- Benchmarks are sequential by design (sandbox loopback throttling makes concurrent HTTP numbers meaningless)

Conclusion

The project demonstrates the core of a production storage architecture: write amortization through sequential I/O, immutability enabling cheap snapshots and compaction, probabilistic membership filters to protect the read path, and ordered iteration for range queries. Both major defects were found by tests and benchmarks rather than by inspection - which is the argument for building the measurement harness alongside the engine.